

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM
KHOA TOÁN - TIN**

ĐỒ ÁN MÔN HỌC

ĐỀ TÀI

**ỨNG DỤNG KHAI PHÁP LUẬT KẾT HỢP (ASSOCIATION
RULES)**

PHÂN TÍCH HÀNH VI MUA SẮM CỦA KHÁCH HÀNG

(Đề tài số 19)

Sinh viên thực hiện:

Lớp:

Giảng viên hướng dẫn:

Đà Nẵng – 2026

MỤC LỤC

MỞ ĐẦU.....	3
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT.....	4
1.1. Khai phá dữ liệu và bài toán phân tích giỏ hàng thị trường.....	4
1.2. Các khái niệm cơ bản.....	4
1.3. Các độ đo đánh giá luật kết hợp.....	5
1.4. Thuật toán vét cạn (brute-force).....	6
1.5. Thuật toán Apriori.....	7
1.6. Thuật toán FP-Growth (kỹ thuật chưa học).....	7
1.7. Từ tập phổ biến đến luật kết hợp và ứng dụng.....	8
CHƯƠNG 2. PHÁT BIỂU VẤN ĐỀ VÀ CÀI ĐẶT THUẬT TOÁN.....	8
2.1. Phát biểu bài toán.....	8
2.2. Cài đặt thuật toán brute-force.....	9
2.3. Cài đặt thuật toán Apriori.....	11
2.4. Sinh luật kết hợp và kiểm chứng với giáo trình.....	14
2.5. FP-Growth với thư viện mlxtend.....	14
2.6. Nhận xét về độ phức tạp.....	15
CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ỨNG DỤNG.....	16
3.1. Bộ dữ liệu thực nghiệm.....	16
3.2. Tiền xử lý dữ liệu.....	16
3.3. Khám phá dữ liệu.....	17
3.4. Xây dựng giỏ hàng và ma trận one-hot.....	18
3.5. Khai phá tập phổ biến và so sánh hiệu năng ba thuật toán.....	18

3.6. Kết quả luật kết hợp.....	20
3.7. Ý nghĩa kinh doanh và đề xuất ứng dụng.....	21
3.8. Đánh giá kết quả và hạn chế.....	21
KẾT LUẬN.....	22
TÀI LIỆU THAM KHẢO.....	22
PHỤ LỤC.....	23
Phụ lục A. Hướng dẫn cài đặt và chạy chương trình.....	23
Phụ lục B. Mã nguồn cài đặt thuật toán (thuat_toan.py).....	25

Ghi chú: Mục lục được sinh tự động. Sau khi chỉnh sửa, nhấp chuột phải vào Mục lục và chọn Update Field để cập nhật số trang.

MỞ ĐẦU

Trong hai thập kỷ gần đây, thương mại điện tử và hệ thống bán lẻ hiện đại đã ghi lại một lượng dữ liệu giao dịch khổng lồ: mỗi đơn hàng, mỗi hóa đơn là một “giỏ hàng” chứa đựng hành vi lựa chọn thực sự của khách hàng. Việc khai thác dữ liệu này để trả lời câu hỏi “khách hàng thường mua những mặt hàng nào cùng nhau?” có ý nghĩa trực tiếp đối với việc trưng bày sản phẩm, xây dựng combo khuyến mãi, thiết kế hệ thống gợi ý và quản lý tồn kho. Khai phá luật kết hợp (association rule mining), được Agrawal và cộng sự giới thiệu từ năm 1993 [1], cho đến nay vẫn là công cụ kinh điển và hiệu quả nhất cho bài toán phân tích giỏ hàng thị trường (market basket analysis).

Đề tài **Ứng dụng khai phá luật kết hợp (association rules) phân tích hành vi mua sắm của khách hàng** được thực hiện nhằm vận dụng kiến thức đã học về thuật toán và trí tuệ nhân tạo vào một bài toán khai phá dữ liệu thực tiễn. Theo yêu cầu của đề tài, đồ án tập trung vào ba trọng tâm: (1) tự cài đặt hai thuật toán khai phá tập phổ biến đã học trong chương trình là **thuật toán vét cạn (brute-force)** và **thuật toán Apriori**; (2) tìm hiểu và áp dụng một kỹ thuật chưa học là **thuật toán FP-Growth**, sử dụng thư viện mã nguồn mở mlxtend [4] do thuật toán có cấu trúc cây FP khá phức tạp; (3) áp dụng các thuật toán trên một bộ dữ liệu giao dịch bán lẻ thực tế, phân tích, trực quan hóa và đánh giá ý nghĩa kinh doanh của các luật tìm được.

Đối tượng nghiên cứu của đề tài là các thuật toán khai phá tập phổ biến và sinh luật kết hợp. **Phạm vi thực nghiệm** là bộ dữ liệu Online Retail của UCI Machine Learning Repository [5], gồm 541.909 dòng giao dịch của một cửa hàng bán lẻ trực tuyến tại Anh trong giai đoạn 12/2010 - 12/2011.

Sản phẩm của đồ án gồm: chương trình Python tự cài đặt đầy đủ hai thuật toán đã học với khả năng truy vết từng bước thực hiện; quy trình xử lý và khai phá trên dữ liệu thực với các bước kiểm chứng chéo; báo cáo trình bày cơ sở lý thuyết, quá trình cài đặt, kết quả thực nghiệm và các đề xuất ứng dụng kinh doanh. Bài báo cáo gồm ba chương: Chương 1 trình bày cơ sở lý thuyết về khai phá luật kết hợp và các thuật toán; Chương 2 phát biểu bài toán, mô tả chi tiết quá trình cài đặt và minh họa từng bước trên dữ liệu mẫu; Chương 3 trình bày kết quả thực nghiệm trên dữ liệu thật, trực quan hóa, đánh giá và đề xuất ứng dụng.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Khai phá dữ liệu và bài toán phân tích giỏ hàng thị trường

Khai phá dữ liệu (data mining) là bước trung tâm của quá trình khám phá tri thức trong dữ liệu (KDD - Knowledge Discovery in Databases), gồm các bước: thu thập và làm sạch dữ liệu, tích hợp dữ liệu, chọn và biến đổi dữ liệu, khai phá tri thức, đánh giá mẫu tin và trình bày tri thức [2]. Trong lĩnh vực thương mại, một trong những bài toán khai phá dữ liệu cổ điển và có giá trị ứng dụng cao nhất là phân tích giỏ hàng thị trường: từ tập hợp các hóa đơn, tìm ra những nhóm mặt hàng thường được mua cùng nhau và biểu diễn chúng dưới dạng các luật dạng “nếu - thì”.

Một luật kết hợp điển hình có dạng $X \rightarrow Y$, đọc là “nếu khách hàng mua tập mặt hàng X thì có xu hướng mua thêm mặt hàng (hoặc tập mặt hàng) Y ”. Ví dụ, luật “bộ chén trà Regency màu xanh $\rightarrow$ bộ chén trà Regency màu hồng” gợi ý rằng hai mẫu của cùng một dòng sản phẩm thường được mua chung. Các luật như vậy là cơ sở cho rất nhiều hoạt động nghiệp vụ: sắp xếp sản phẩm liên kề trên kệ hoặc trên trang web, đóng gói combo giảm giá, gợi ý sản phẩm bổ sung khi khách hàng xem một sản phẩm, dự báo nhu cầu giữa các nhóm hàng liên quan.

1.2. Các khái niệm cơ bản

Cho tập hợp $I = \{i_1, i_2, \dots, i_m\}$ gồm m mặt hàng (item). Một giao dịch (transaction) T là một tập con của I đại diện cho một giỏ hàng; cơ sở dữ liệu giao dịch D là tập hợp các giao dịch $D = \{T_1, T_2, \dots, T_n\}$. Một tập mặt hàng (itemset) X là tập con bất kỳ của I ; tập có k phần tử được gọi là tập k -mặt hàng (k -itemset).

Tập mặt hàng X được gọi là **tập phổ biến (frequent itemset)** nếu độ hỗ trợ của X không nhỏ hơn ngưỡng min_support do người dùng định trước. **Luật kết hợp** là một mệnh đề có dạng $X \rightarrow Y$ với $X \subset I, Y \subset I, X \cap Y = \emptyset, X \neq \emptyset, Y \neq \emptyset$. Bài toán khai phá luật kết hợp gồm hai giai đoạn: (1) tìm toàn bộ các tập phổ biến với độ hỗ trợ $\geq \text{min_support}$; (2) từ các tập phổ biến đó sinh ra các luật có độ tin cậy $\geq \text{min_confidence}$ [1]. Giai đoạn (1) là nút thắt cổ chai về tính toán, do đó các thuật toán trong đề tài đều tập trung vào giai đoạn này.

1.3. Các độ đo đánh giá luật kết hợp

Gọi $T(X)$ là tập các giao dịch chứa tập mặt hàng X , N là số giao dịch của D . Các độ đo được sử dụng trong đề tài được định nghĩa như sau [2], [6]:

Độ hỗ trợ (support) đo mức phổ biến của tập mặt hàng trong toàn bộ dữ liệu:

$$sup(X) = |T(X)| / |T| \quad (1.1)$$

Độ tin cậy (confidence) là xác suất có điều kiện $P(Y | X)$: trong những giao dịch chứa X , bao nhiêu phần trăm chứa thêm Y .

$$conf(X \rightarrow Y) = sup(X \cup Y) / sup(X) \quad (1.2)$$

Độ nâng (lift) cho biết sự xuất hiện của X làm tăng xác suất mua Y bao nhiêu lần; lift > 1 nghĩa là X và Y có tương quan dương, lift $= 1$ là độc lập, lift < 1 là tương quan âm.

$$lift(X \rightarrow Y) = conf(X \rightarrow Y) / sup(Y) \quad (1.3)$$

Ngoài hai độ đo cốt lõi, đề tài còn tính hai độ đo bổ sung để đánh giá sâu hơn chất lượng luật: **leverage** đo phần chênh lệch giữa tần suất đồng xuất hiện thực tế và trường hợp độc lập, còn **conviction** đo mức “vi phạm dự kiến” của luật - conviction càng lớn thì luật càng ít bị phủ nhận.

$$lev(X \rightarrow Y) = sup(X \cup Y) - sup(X) \cdot sup(Y) \quad (1.4)$$

$$conv(X \rightarrow Y) = (1 - sup(Y)) / (1 - conf(X \rightarrow Y)) \quad (1.5)$$

Bảng 1.1. Ý nghĩa của các độ đo và ngưỡng thường dùng

Độ đo	Phạm vi giá trị	Ý nghĩa	Ngưỡng thường dùng
Support	[0, 1]	Mức độ phổ biến của tập mặt hàng	0,01 - 0,05 với dữ liệu lớn
Confidence	[0, 1]	Độ chắc chắn của luật $X \rightarrow Y$	0,3 - 0,7
Lift	[0, $+\infty$)	Mức tương quan giữa X và Y	> 1 (tương quan dương)
Leverage	[-1, 1]	Chênh lệch so với trường hợp độc lập	> 0
Conviction	[0, $+\infty$)	Mức phủ nhận dự kiến của luật	> 1

Cần nhấn mạnh rằng support và confidence là hai độ đo bắt buộc trong định nghĩa bài toán, nhưng luật có confidence cao chưa chắc có ý nghĩa nếu $\text{lift} \leq 1$ (hai mặt hàng thực chất độc lập). Vì vậy trong thực nghiệm, đề tài sử dụng đồng thời ba ngưỡng min_support , min_confidence và min_lift để lọc các luật chất lượng.

1.4. Thuật toán vét cạn (brute-force)

Thuật toán vét cạn là cách tiếp cận trực diện nhất cho bài toán tìm tập phổ biến: sinh **toàn bộ** các tập con khác rỗng của I ($2^m - 1$ tập), quét toàn bộ cơ sở giao dịch để đếm độ hỗ trợ của từng tập, rồi loại bỏ các tập không đạt min_support . Số ứng viên cần xét chỉ tính riêng các tập ≤ 4 phần tử với $m = 2.530$ mặt hàng đã lên tới hơn 1012 - điều này cho thấy độ phức tạp mũ của phương pháp.

Trong chương trình học thuật toán, vét cạn là kỹ thuật nền tảng, được dùng làm “chuẩn so sánh” (baseline) cho tính đúng đắn: kết quả do các thuật toán tối ưu hơn sinh ra phải trùng khớp hoàn toàn với kết quả vét cạn trên dữ liệu nhỏ. Trong đồ án, brute-force được cài đặt đầy đủ và chạy minh họa trên dữ liệu mẫu 9 giao dịch, đồng thời là điểm tựa để kiểm chứng Apriori.

1.5. Thuật toán Apriori

Apriori [1] là thuật toán kinh điển cho bài toán khai phá tập phổ biến, dựa trên **tính chất Apriori**: mọi tập con của một tập phổ biến đều phổ biến; phát biểu ngược lại, nếu X không phổ biến thì mọi tập lớn hơn chứa X đều không phổ biến. Tính chất cho phép **cắt tỉa (prune)** một lượng lớn ứng viên mà không cần đếm. Thuật toán lặp qua các cấp $k = 1, 2, \dots$ với ba bước chính tại mỗi cấp [2]:

- **Join (kết nối)**: sinh tập ứng viên C_k độ dài k bằng cách kết nối các cặp tập phổ biến $(k-1)$ -mặt hàng của F_{k-1} có chung $(k-2)$ phần tử đầu tiên;
- **Prune (cắt tỉa)**: loại khỏi C_k mọi ứng viên có ít nhất một tập con $(k-1)$ -mặt hàng không thuộc F_{k-1} ;
- **Quét (scan)**: duyệt toàn bộ giao dịch, đếm hỗ trợ của các ứng viên còn lại, giữ lại $F_k = \{X \in C_k \mid \text{sup}(X) \geq \text{min_support}\}$.

Lặp dừng khi $F_k = \emptyset$. So với vét cạn, Apriori giảm mạnh không gian tìm kiếm nhờ prune và nhờ chỉ xét các “mở rộng” của tập phổ biến hiện có; tuy nhiên vẫn phải

quét dữ liệu nhiều lần (số lần quét bằng số cấp k) và sinh nhiều ứng viên trung gian ở các cấp đầu.

1.6. Thuật toán FP-Growth (kỹ thuật chưa học)

FP-Growth (Frequent Pattern Growth) [3] là kỹ thuật khai phá tập phổ biến **không sinh ứng viên**: dữ liệu được nén vào một cấu trúc cây gọi là **cây FP (FP-tree)** - mỗi nút lưu một mặt hàng kèm bộ đếm, các giao dịch có phần đầu chung đi chung nhánh, các mặt hàng trong mỗi giao dịch được sắp theo thứ tự tần suất giảm dần, kèm bảng tiêu đề (header table) nối các nút cùng mặt hàng bằng danh sách liên kết. Sau khi dựng cây với **hai lần quét dữ liệu**, thuật toán khai phá đệ quy bằng cách xây “cây FP có điều kiện” (conditional FP-tree) cho từng mặt hàng ở cuối bảng tiêu đề.

So với Apriori, FP-Growth tránh được chi phí sinh - kiểm tra ứng viên và chỉ quét dữ liệu hai lần, thường nhanh hơn rõ rệt trên dữ liệu dày đặc. Tuy nhiên cấu trúc cây FP và thủ tục khai phá đệ quy khá phức tạp để cài đặt đúng ngay lần đầu. Theo đúng tinh thần yêu cầu của đề tài (“có thể dùng thư viện có sẵn nếu thuật toán quá phức tạp”), đồ án sử dụng cài đặt FP-Growth hiệu năng cao trong thư viện mlxtend [4], và dùng kết quả của nó để **kiểm chứng chéo** với Apriori tự cài đặt.

1.7. Từ tập phổ biến đến luật kết hợp và ứng dụng

Với mỗi tập phổ biến X có $|X| \geq 2$, sinh tất cả các phân hoạch $X = A \cup B$ với $A \neq \emptyset$, $B \neq \emptyset$, tạo luật $A \rightarrow B$, rồi tính các độ đo theo công thức (1.1) - (1.5) và lọc theo ngưỡng. Số luật tiềm năng tăng theo cấp số nhân với độ dài tập phổ biến, nên trong thực nghiệm đề tài giới hạn ở các tập có độ dài ≤ 3 , vừa đủ diễn giải kinh doanh (luật “hai vế” dễ ứng dụng hơn luật nhiều vế). Các luật chất lượng cao chính là đầu vào của các ứng dụng: hệ thống gợi ý dạng “khách mua A thường mua B ”, xây dựng combo/khuyến mãi chéo, sắp xếp vị trí trưng bày và quản lý tồn kho theo cụm sản phẩm liên quan.

CHƯƠNG 2. PHÁT BIỂU VẤN ĐỀ VÀ CÀI ĐẶT THUẬT TOÁN

2.1. Phát biểu bài toán

Đầu vào: cơ sở dữ liệu giao dịch D gồm n giỏ hàng, mỗi giỏ là một tập mặt hàng; hai tham số min_support và min_confidence . **Đầu ra:** toàn bộ tập phổ biến F với $\text{sup} \geq \text{min_support}$ và toàn bộ luật kết hợp $X \rightarrow Y$ với $\text{conf} \geq \text{min_confidence}$, $\text{lift} > 1$. Bài toán được chia thành ba thuật toán con theo đúng yêu cầu của đề tài: (1) brute-force - vét cạn toàn bộ tập con; (2) Apriori - join, prune, quét; (3) FP-Growth - dùng thư viện. Hai thuật toán tự cài đặt được chạy **theo từng bước** trên bộ dữ liệu mẫu nhỏ để truy vết đầy đủ quá trình tính toán.

Bộ dữ liệu mẫu sử dụng là ví dụ kinh điển AllElectronics trong giáo trình Data Mining: Concepts and Techniques của Han và Kamber [2], gồm 9 giao dịch, 5 mặt hàng I1 - I5. Chọn ngưỡng $\text{min_support} = 2/9 \approx 22,2\%$ (tức số giao dịch tối thiểu là 2) và $\text{min_confidence} = 2/3 \approx 66,7\%$ - trùng với ví dụ trong giáo trình để có “đáp án chuẩn” kiểm chứng.

Bảng 2.1. Bộ dữ liệu mẫu 9 giao dịch (theo giáo trình Han và Kamber [2])

Giao dịch	Các mặt hàng	Giao dịch	Các mặt hàng
T1	I1, I2, I5	T6	I2, I3
T2	I2, I4	T7	I1, I3
T3	I2, I3	T8	I1, I2, I3, I5
T4	I1, I2, I4	T9	I1, I2, I3
T5	I1, I3		

2.2. Cài đặt thuật toán brute-force

Cài đặt sinh toàn bộ tập con theo từng cấp k bằng phép tổ hợp, sau đó quét dữ liệu đếm hỗ trợ và lọc theo ngưỡng. Đoạn mã lõi (toàn bộ mã nguồn ở Phụ lục B):

```
def brute_force(giao_dich, min_sup, k_max=None, trace=None):
    n = len(giao_dich)
    phan_tu = sorted({item for t in giao_dich for item in t})
    ket_qua = {}
    for k in range(1, k_max + 1):
        # Bước 1: sinh TẤT CẢ tập con k phần tử (không có prune)
        ung_vien = [frozenset(c) for c in combinations(phan_tu, k)]
        # Bước 2: quét đếm support từng ứng viên
        dem = do_support_quet(giao_dich, ung_vien, n)
        # Bước 3: lọc bởi min_support
```

```

L_k = {u: c for u, c in dem.items() if c / n >= min_sup}
if not L_k: break
ket_qua[k] = L_k
return ket_qua

```

Kết quả chạy từng bước trên dữ liệu mẫu được tổng hợp ở Bảng 2.2. Tổng số ứng viên phải đếm là $5 + 10 + 10 + 5 = 30$ tập (chưa kể tập 5 phần tử), trong khi Apriori chỉ cần đếm $5 + 10 + 2 = 17$ tập - minh chứng rõ rệt cho hiệu quả cắt tỉa.

Bảng 2.2. Brute-force chạy từng bước trên dữ liệu mẫu ($\text{min_support} = 2/9$)

Cấp k	Số ứng viên sinh	Số tập phổ biến	Các tập phổ biến (số giao dịch)
1	5	5	I1(6), I2(7), I3(6), I4(2), I5(2)
2	10	6	I1I2(4), I1I3(4), I1I5(2), I2I3(4), I2I4(2), I2I5(2)
3	10	2	I1I2I3(2), I1I2I5(2)
4	5	0	-

2.3. Cài đặt thuật toán Apriori

Cài đặt Apriori tuân thủ đúng ba bước join - prune - quét. Đoạn mã lõi của hai bước join và prune:

```

# Bước 2 (JOIN): sinh C(k) từ F(k-1)
tap_sap_xep = sorted([sorted(u) for u in tap_F_km1])
ung_vien = set()
for i in range(len(tap_sap_xep)):
    for j in range(i + 1, len(tap_sap_xep)):
        a, b = tap_sap_xep[i], tap_sap_xep[j]
        if a[:-1] == b[:-1]: # giống nhau k-2 phần tử đầu
            ung_vien.add(frozenset(a) | frozenset(b))

# Bước 3 (PRUNE): loại ứng viên có tập con không phổ biến
C_k, bi_cat = [], []
for u in ung_vien:
    if all(frozenset(sub) in tap_F_km1
           for sub in combinations(u, k - 1)):
        C_k.append(u)
    else:
        bi_cat.append(u) # bị cắt bởi tính chất Apriori

```

Kết quả chạy từng bước được tổng hợp ở Bảng 2.3. Điểm đáng chú ý: tại cấp 3, bước join sinh 6 ứng viên nhưng bước prune loại ngay 4 ứng viên ($\{I1, I3, I5\}$, $\{I2, I3, I4\}$, $\{I2, I3, I5\}$, $\{I2, I4, I5\}$) do chứa tập con không phổ biến ($\{I3, I5\}$ chỉ xuất hiện 1 lần,

{I3,I4} và {I4,I5} không xuất hiện); chỉ còn 2 ứng viên cần quét đếm. Tại cấp 4, ứng viên duy nhất {I1,I2,I3,I5} cũng bị cắt hoàn toàn.

Bảng 2.3. Apriori chạy từng bước trên dữ liệu mẫu (min_support = 2/9)

Cấp k	Join sinh	Prune cắt	Quét đếm	Tập phổ biến
1	5	0	5	I1, I2, I3, I4, I5
2	10	0	10	6 cặp (xem Bảng 2.2)
3	6	4	2	I1I2I3(2), I1I2I5(2)
4	1	1	0	-

2.4. Sinh luật kết hợp và kiểm chứng với giáo trình

Từ các tập phổ biến, chương trình sinh tất cả các luật $A \rightarrow B$ (A là tập con khác rỗng của tập phổ biến) và lọc theo min_confidence = 2/3, min_lift = 1. Kết quả thu được 8 luật, trình bày ở Bảng 2.4.

Bảng 2.4. 8 luật kết hợp tìm được trên dữ liệu mẫu (conf $\geq$ 2/3, lift $\geq$ 1)

Luật	Support	Confidence	Lift	Leverage
I5 $\rightarrow$ I1, I2	0,222	1,000	2,250	0,123
I5 $\rightarrow$ I1	0,222	1,000	1,500	0,074
I2, I5 $\rightarrow$ I1	0,222	1,000	1,500	0,074
I5 $\rightarrow$ I2	0,222	1,000	1,286	0,049
I4 $\rightarrow$ I2	0,222	1,000	1,286	0,049
I1, I5 $\rightarrow$ I2	0,222	1,000	1,286	0,049
I1 $\rightarrow$ I3	0,444	0,667	1,000	0,000
I3 $\rightarrow$ I1	0,444	0,667	1,000	0,000

So sánh với kết quả chuẩn trong giáo trình [2]: L1 gồm cả 5 mặt hàng; L2 gồm 6 cặp; L3 gồm {I1, I2, I3} và {I1, I2, I5} với số giao dịch lần lượt trùng khớp. Chương trình kiểm chứng tự động phần này và kết luận **khớp 100%** với đáp án chuẩn - xác nhận cả hai thuật toán tự cài đặt đúng đắn. Cũng cần lưu ý 2 luật cuối (I1 $\rightarrow$ I3 và I3 $\rightarrow$ I1) có lift đúng bằng 1: hai mặt hàng thực chất độc lập, minh chứng cho việc vì sao cần thêm ngưỡng lift khi lọc luật.

2.5. FP-Growth với thư viện mlxtend

Giai đoạn khai phá tập phổ biến bằng FP-Growth được thực hiện qua thư viện mlxtend [4]: dữ liệu được mã hóa one-hot bằng TransactionEncoder, sau đó gọi hàm fpgrowth. Đoạn mã sử dụng:

```
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth

te = TransactionEncoder()
onehot = pd.DataFrame(te.fit(gio_hang).transform(gio_hang),
                      columns=te.columns_)
frequent = fpgrowth(onehot, min_support=0.02, use_colnames=True)
```

Kết quả của FP-Growth sẽ được so sánh từng phần tử với kết quả Apriori tự cài đặt trên dữ liệu thực ở Chương 3 (mục 3.5) - đây chính là kiểm chứng chéo giữa hai thuật toán được cài đặt bằng hai cách hoàn toàn độc lập.

2.6. Nhận xét về độ phức tạp

Bảng 2.5. So sánh lý thuyết ba thuật toán

Thuật toán	Ý tưởng chính	Số lần quét dữ liệu	Ưu / nhược điểm
Brute-force	Vét toàn bộ $2^m - 1$ tập con	mỗi ứng viên 1 lần quét	Đơn giản, đúng đắn / độ phức tạp mũ, chỉ dùng cho dữ liệu nhỏ
Apriori	Join + prune theo tính chất Apriori	bằng số cấp k	Giảm mạnh ứng viên / vẫn quét nhiều lần, sinh nhiều ứng viên cấp thấp
FP-Growth	Dựng cây FP, khai phá đệ quy	2	Không sinh ứng viên, nhanh trên dữ liệu dày đặc / cấu trúc phức tạp, tốn bộ nhớ cây

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ỨNG DỤNG

3.1. Bộ dữ liệu thực nghiệm

Đề tài sử dụng bộ dữ liệu **Online Retail** công khai trên UCI Machine Learning Repository [5]: dữ liệu giao dịch thực của một cửa hàng bán lẻ trực tuyến tại Anh, từ 01/12/2010 đến 09/12/2011, gồm 541.909 dòng và 8 thuộc tính. Bộ dữ liệu này thường được dùng làm chuẩn cho bài toán phân tích giỏ hàng vì có quy mô lớn, có các “nhiều” thực tế (đơn hủy, giá trị âm, thiếu thông tin) để thực hành tiền xử lý.

Bảng 3.1. Mô tả các thuộc tính của bộ dữ liệu Online Retail

Thuộc tính	Kiểu dữ liệu	Ý nghĩa
InvoiceNo	Chuỗi	Mã đơn hàng (6 số); đơn hủy bắt đầu bằng chữ C
StockCode	Chuỗi	Mã sản phẩm
Description	Chuỗi	Tên/mô tả sản phẩm
Quantity	Số nguyên	Số lượng sản phẩm trong dòng giao dịch
InvoiceDate	Thời gian	Ngày và giờ tạo đơn hàng
UnitPrice	Số thực	Đơn giá (bảng Anh, pound)
CustomerID	Số thực	Mã khách hàng (có thể thiếu)
Country	Chuỗi	Quốc gia của khách hàng

3.2. Tiền xử lý dữ liệu

Quy trình làm sạch được thiết kế để mỗi dòng còn lại là một giao dịch mua sản phẩm thực sự:

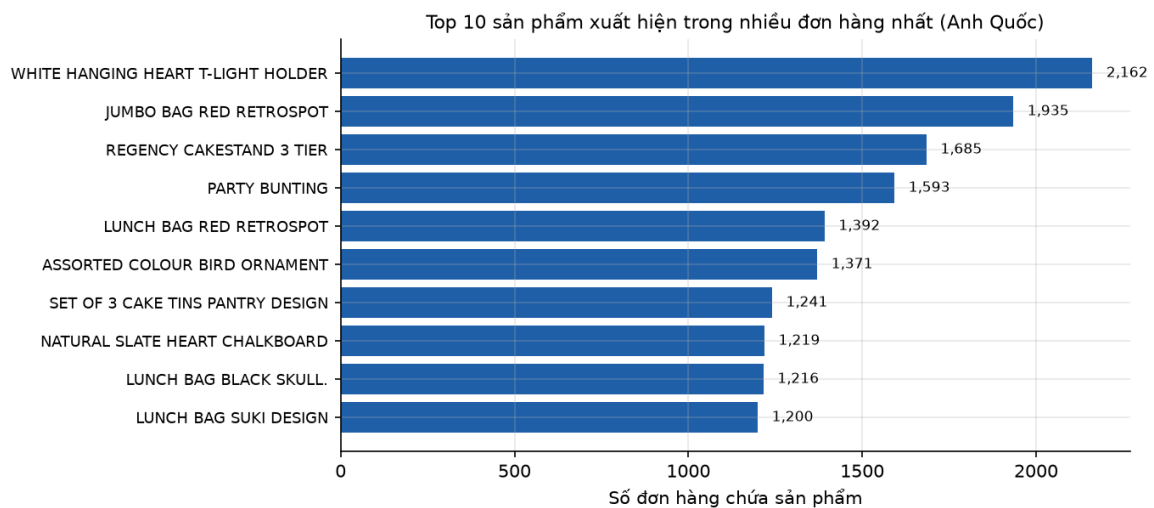
- Loại các dòng thiếu mô tả sản phẩm (Description rỗng);
- Loại toàn bộ đơn hàng hủy/trả hàng - mã đơn bắt đầu bằng chữ C, các dòng này có Quantity âm;
- Giữ lại chỉ các dòng có Quantity > 0 và UnitPrice > 0;
- Chuẩn hóa tên sản phẩm (in hoa, cắt khoảng trắng thừa) và loại các khoản thu dịch vụ không phải sản phẩm (postage, carriage, bank charges, manual, samples...).

Sau làm sạch còn lại **527.634 dòng** (giảm 14.275 dòng, khoảng 2,6%), thuộc **19.773 đơn hàng** và 3.997 mặt hàng. Do các đơn hàng quốc tế phân tán ở 38 quốc gia

nhưng chiếm tỷ lệ nhỏ, đề tài chọn phân tích chính trên thị trường **Anh Quốc** - chiếm 91,7% số dòng (483.884 dòng) - để bảo đảm mỗi cấu trúc luật đều đủ dữ liệu thống kê.

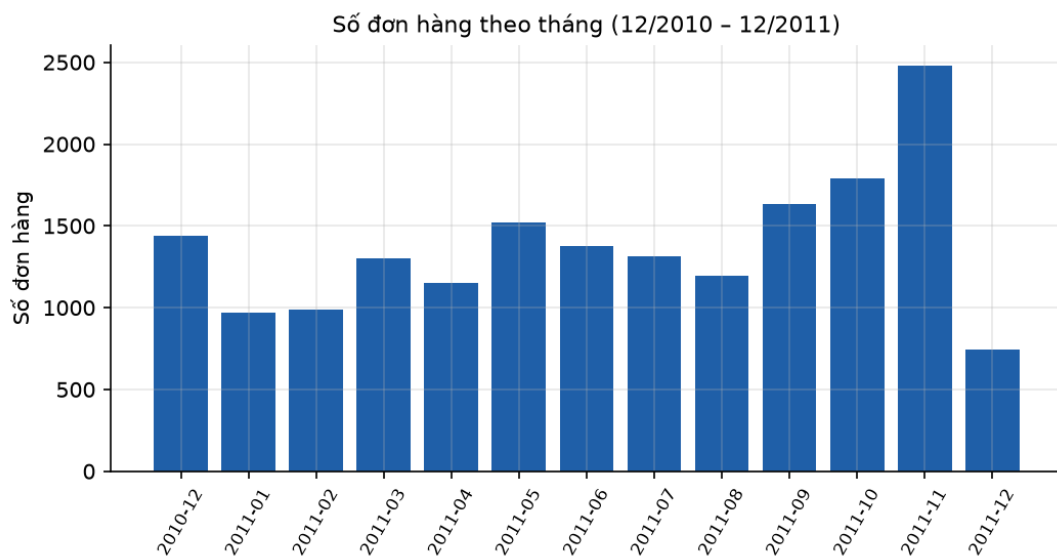
3.3. Khám phá dữ liệu

Hình 3.1 cho thấy 10 sản phẩm xuất hiện trong nhiều đơn hàng nhất. Nhóm dẫn đầu gồm các món đồ trang trí nhà sinh hoạt giá nhỏ (đế đèn treo hình trái tim WHITE HANGING HEART T-LIGHT HOLDER xuất hiện trong 2.162 đơn, túi JUMBO BAG RED RETROSPOT trong 1.935 đơn, đế bánh REGENCY CAKESTAND 3 TIER trong 1.685 đơn). Đây đều là sản phẩm có giá bán nhỏ, mua thường xuyên hoặc mua kèm nhiều - đặc điểm điển hình của mặt hàng “sát dân” trong bán lẻ quà tặng.



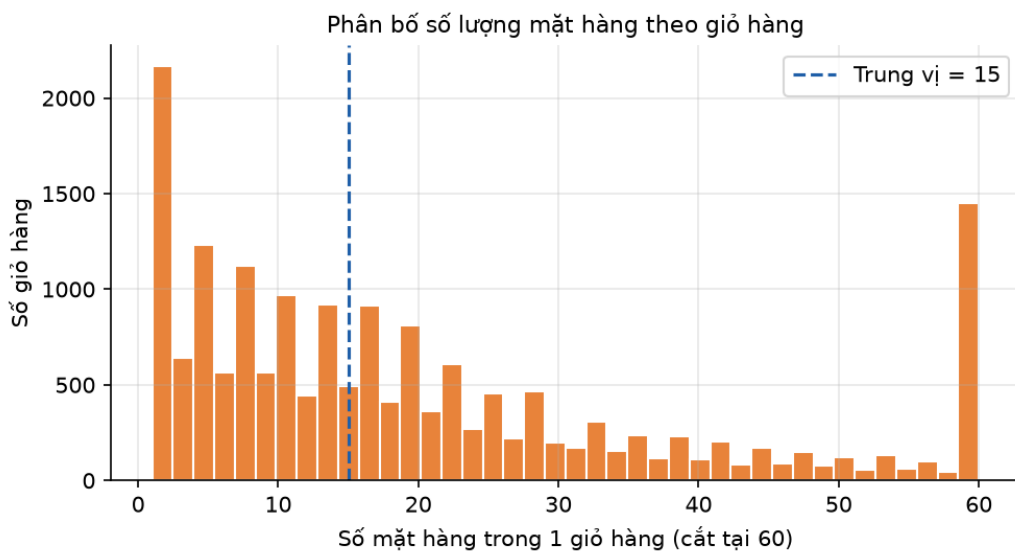
Hình 3.1. Top 10 sản phẩm xuất hiện trong nhiều đơn hàng nhất (Anh Quốc)

Hình 3.2 thể hiện tính mùa vụ rõ rệt của hoạt động bán hàng: số đơn hàng tăng mạnh từ tháng 8 và đạt đỉnh vào tháng 11 (gần 2.500 đơn), phù hợp với mùa chuẩn bị Giáng sinh - mùa mua sắm cao điểm của ngành quà tặng trang trí ở Anh. Tháng 12/2011 chỉ còn dữ liệu 9 ngày đầu tháng nên thấp bất thường.



Hình 3.2. Số đơn hàng theo tháng (12/2010 - 12/2011)

Hình 3.3 mô tả phân bố số mặt hàng theo giỏ hàng: trung vị là 15 mặt hàng/giỏ, trung bình 26,4. Phân phối lệch phải rõ rệt - đa số đơn hàng vừa và nhỏ, song tồn tại các đơn sỉ rất lớn (lên tới hàng trăm mặt hàng khác nhau) của khách mua đi buôn, một đặc thù của dữ liệu bán buôn - bán lẻ trộn lẫn.



Hình 3.3. Phân bố số lượng mặt hàng theo giỏ hàng

3.4. Xây dựng giỏ hàng và ma trận one-hot

Mỗi đơn hàng (InvoiceNo) được gộp thành một giỏ hàng - tập hợp các tên sản phẩm không trùng lặp. Với 17.901 giỏ hàng của thị trường Anh, đề tài áp dụng thêm hai bước thu gọn hợp lý cho bài toán luật kết hợp: (1) loại các mặt hàng hiếm - chỉ giữ mặt

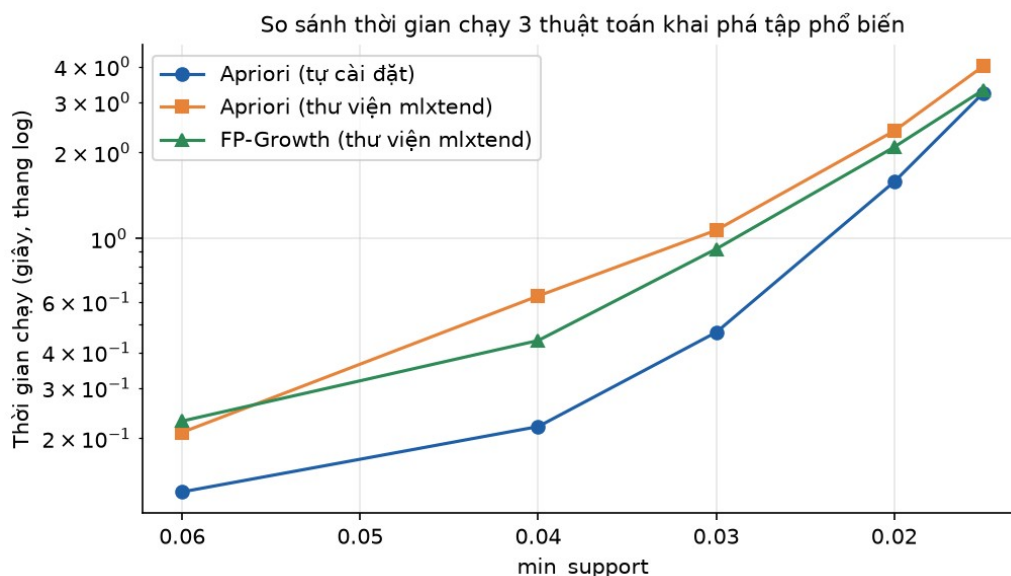
hàng xuất hiện trong ít nhất 30 giỏ hàng (khoảng 0,17%) vì luật từ mặt hàng quá hiếm không đủ dữ liệu thống kê và ít giá trị hành động; (2) loại giỏ hàng có ít hơn 2 mặt hàng vì không sinh được luật. Kết quả thu được **16.406 giỏ hàng** và **2.530 mặt hàng**. Toàn bộ giỏ hàng được mã hóa thành ma trận one-hot 16.406×2.530 bằng TransactionEncoder - mỗi ô là 1 nếu mặt hàng có mặt trong giỏ.

3.5. Khai phá tập phổ biến và so sánh hiệu năng ba thuật toán

Ba thuật toán được chạy trên cùng dữ liệu tại 5 mức min_support giảm dần từ 6% xuống 1,5%. Kết quả ở Bảng 3.2 cho thấy: **số tập phổ biến sinh ra bởi Apriori tự cài đặt và FP-Growth của thư viện trùng khớp tuyệt đối ở mọi mức ngưỡng** (26, 87, 181, 483, 943 tập). Đây là kiểm chứng chéo mạnh giữa hai cài đặt độc lập, khẳng định độ tin cậy của chương trình tự viết.

Bảng 3.2. Số tập phổ biến và thời gian chạy của ba thuật toán (Python 3.12, máy cá nhân)

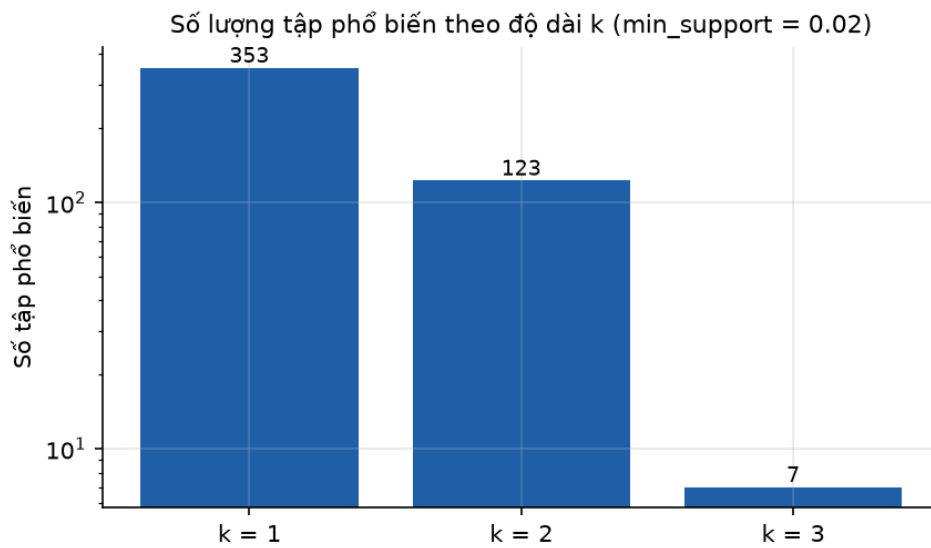
min_support	Số tập phổ biến	Apriori tự cài đặt (giây)	Apriori mlxtend (giây)	FP-Growth (giây)
0,06	26	0,15	0,23	0,39
0,04	87	0,27	0,65	0,66
0,03	181	0,62	1,16	1,01
0,02	483	1,60	2,29	2,37
0,015	943	3,04	3,92	3,16



Hình 3.4. So sánh thời gian chạy ba thuật toán khai phá tập phổ biến

Điều thú vị là phiên bản Apriori tự cài đặt (dùng chỉ mục ngược item → danh sách giao dịch, đếm hỗ trợ bằng phép giao tập) chạy **nhANH hơn cả hai cài đặt của thư viện** ở hầu hết các ngưỡng: tại min_support = 2% đạt 1,60 giây so với 2,29 giây (Apriori mlxtend) và 2,37 giây (FP-Growth mlxtend). Nguyên nhân là cách đếm theo giao danh sách (tid-list) chỉ “chạm” đúng những giao dịch có liên quan, trong khi thư viện hoạt động trên ma trận one-hot dày đặc và phải trả thêm chi phí khung tổng quát. Nhận xét này cũng minh họa một bài học kỹ thuật: lựa chọn cấu trúc dữ liệu cho bước đếm hỗ trợ quan trọng không kém thuật toán.

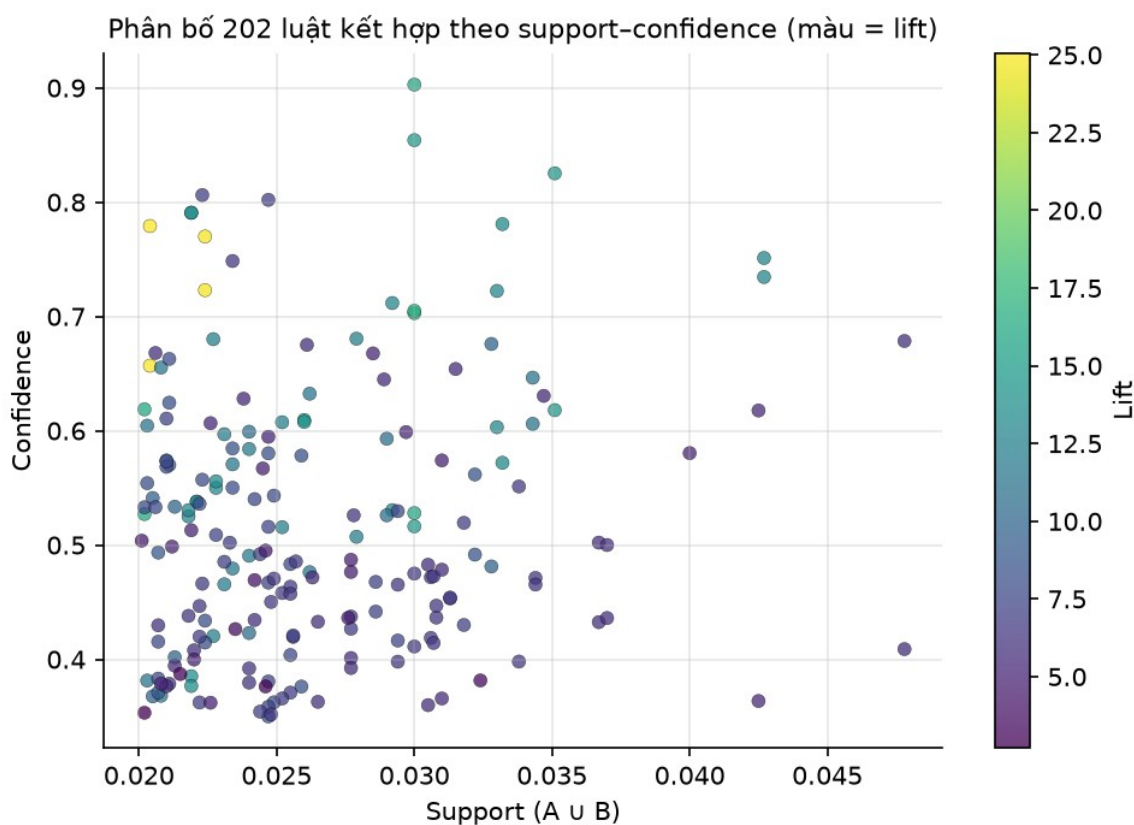
Chọn min_support = 2% (tương đương 328 giỏ hàng) làm ngưỡng phân tích chính - đủ chặt để luật có ý nghĩa thống kê, đủ lỏng để còn lại các cụm sản phẩm thú vị. Kết quả gồm 483 tập phổ biến, phân bố theo độ dài ở Hình 3.5: 353 tập 1-mặt hàng, 123 tập 2-mặt hàng và 7 tập 3-mặt hàng; không còn tập 4-mặt hàng ở ngưỡng này.



Hình 3.5. Số lượng tập phổ biến theo độ dài k (min_support = 2%)

3.6. Kết quả luật kết hợp

Từ 483 tập phổ biến, chương trình sinh luật và lọc theo min_confidence = 35% và min_lift = 1,2, thu được **202 luật kết hợp** (so khớp 100% với bộ luật sinh bởi thư viện mlxtend). Hình 3.6 cho thấy phân bố các luật trong không gian support - confidence: các luật có lift cao (màu sáng) tập trung ở vùng support thấp - confidence cao, còn các luật support cao có lift ở mức trung bình 5 - 8. Không có luật nào rơi vào vùng support và confidence cùng cao - phản ánh đặc điểm bán lẻ quà tặng: hiếm có cặp sản phẩm nào “luôn đi cùng nhau” trong đa số đơn hàng, nhưng tồn tại nhiều quan hệ tương quan rất mạnh trong các nhóm khách mua có chủ đích.



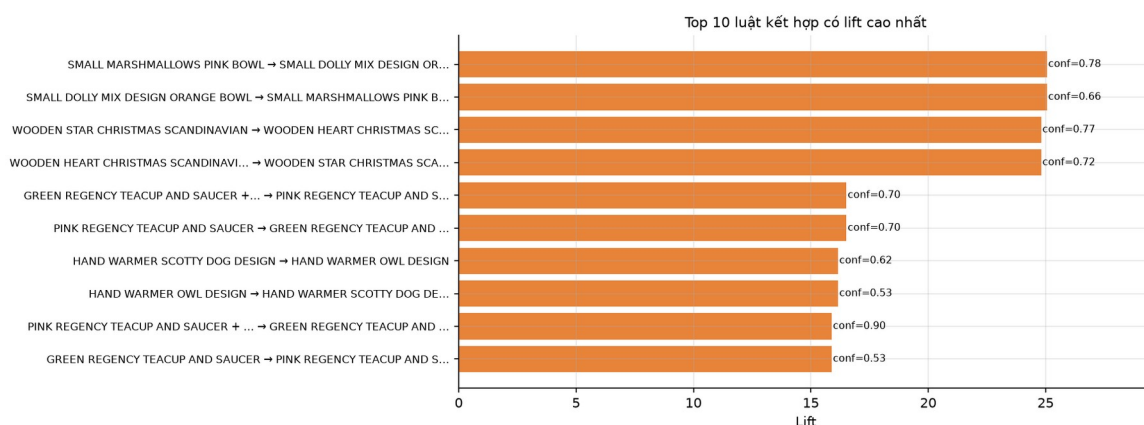
Hình 3.6. Phân bố 202 luật kết hợp theo support - confidence (màu = lift)

Bảng 3.3. Top 10 luật kết hợp có lift cao nhất

Luật kết hợp	Support	Conf	Lift
Bát nhỏ Marshmallow màu hồng → bát nhỏ Dolly Mix màu cam	0,020	0,779	25,06
Bát nhỏ Dolly Mix màu cam → bát nhỏ Marshmallow màu hồng	0,020	0,657	25,06
Sao gỗ Giáng sinh Bắc Âu → tim gỗ Giáng sinh Bắc Âu	0,022	0,770	24,81
Tim gỗ Giáng sinh Bắc Âu → sao gỗ Giáng sinh Bắc Âu	0,022	0,723	24,81
Chén Regency xanh + chén Regency hoa hồng → chén Regency hồng	0,030	0,703	16,52
Chén Regency hồng → chén Regency xanh + chén Regency hoa hồng	0,030	0,705	16,52

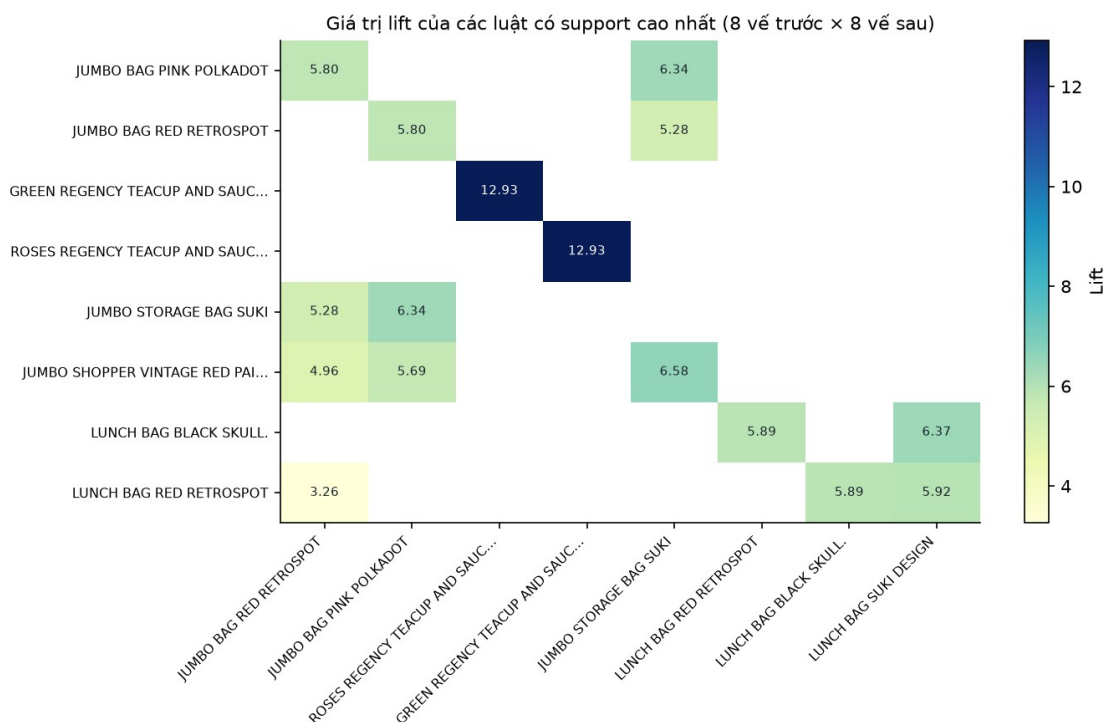
Luật kết hợp	Support	Conf	Lift
Túi sưởi tay họa tiết chó Scotty → túi sưởi tay họa tiết cú mèo	0,020	0,619	16,16
Túi sưởi tay họa tiết cú mèo → túi sưởi tay họa tiết chó Scotty	0,020	0,527	16,16
Chén Regency hồng + chén Regency hoa hồng → chén Regency xanh	0,030	0,903	15,89
Chén Regency xanh → chén Regency hồng + chén Regency hoa hồng	0,030	0,528	15,89

(Tên luật đã được dịch nghĩa từ tên sản phẩm gốc trong dữ liệu, ví dụ: PINK REGENCY TEACUP AND SAUCER = bộ chén và đĩa trà Regency màu hồng; HAND WARMER SCOTTY DOG DESIGN = túi sưởi tay in hình chó Scotty.)



Hình 3.7. Top 10 luật kết hợp có lift cao nhất

Các luật lift cao nhất vẽ nên một bức tranh rất nhất quán: khách hàng có xu hướng mua **cùng lúc nhiều biến thể của cùng một dòng sản phẩm** - đủ bộ ba màu bộ chén Regency, cặp sao - tim gỗ trang trí Giáng sinh phong cách Bắc Âu, hai họa tiết túi sưởi tay, cặp bát kẹo hai màu. Đáng chú ý, luật “chén Regency hồng + chén Regency hoa hồng → chén Regency xanh” có confidence tới 90,3% và conviction 9,7: trong gần mọi giỏ hàng đã có 2 trong 3 màu của bộ chén Regency, khách hàng mua thêm màu xanh. Hình 3.8 bổ sung góc nhìn trên nhóm các luật có support cao nhất (vùng “cột sống” của doanh số): ba cụm JUMBO BAG - LUNCH BAG (các mẫu túi), REGENCY (bộ chén trà) tương quan chặt chẽ trong nội bộ mỗi cụm.



Hình 3.8. Giá trị lift của các luật có support cao nhất (8 vẻ trước × 8 vẻ sau)

3.7. Ý nghĩa kinh doanh và đề xuất ứng dụng

Từ 202 luật thu được, có thể rút ra ba nhóm hành động cụ thể cho cửa hàng:

Bảng 3.4. Phân nhóm luật và đề xuất ứng dụng kinh doanh

Nhóm luật	Bảng chứng tiêu biểu	Ứng dụng đề xuất
Các biến thể cùng dòng sản phẩm (Regency, túi sưỡi, bát kẹo, sao - tim gỗ)	Lift 16 - 25; conf tới 90% khi có 2/3 màu bộ chén	Gợi ý “hoàn thiện bộ sưu tập”: khi khách xem 1 màu, đề xuất ngay các màu còn lại; combo “bộ 3 màu” giảm giá nhẹ
Cụm túi mua sắm JUMBO BAG - LUNCH BAG - đếm các mẫu	Support cao nhất (2 - 4%), lift 3,6 - 7,6	Trưng bày liên kề trên web và tại kho; gói mua 2 - 3 túi khác mẫu; gợi ý thêm vào giỏ theo mẫu
Hàng mùa Giáng sinh (trang trí gỗ Bắc Âu, quà nhỏ)	Luật lift 24,8 giữa sao - tim gỗ; doanh số đạt đỉnh tháng 11	Từ tháng 9 - 10 gom nhóm hàng Noel thành gian riêng; combo trang trí cùng phong cách; nhập tồn theo cụm liên quan

Các luật còn trực tiếp dùng được cho một hệ thống gợi ý sản phẩm dạng luật (“khách mua A thường mua B”) - tương tự mục đích của đề tài số 13 trong danh sách đồ án; với mỗi sản phẩm trong giỏ, hệ thống lấy các luật có vẻ trước chứa sản phẩm đó,

xếp hạng theo $\text{lift} \times \text{confidence}$ và đề xuất về sau. Nhờ luật được khai phá ngoại tuyến (batch), hệ thống gợi ý thời gian thực chỉ cần tra bảng rất nhẹ.

3.8. Đánh giá kết quả và hạn chế

Ưu điểm của quy trình: (1) hai thuật toán trong chương trình được cài đặt hoàn chỉnh, truy vết từng bước và khớp 100% đáp án giáo trình; (2) FP-Growth chưa học được ứng dụng hợp lý qua thư viện và dùng làm kiểm chứng chéo với Apriori tự viết ở mọi ngưỡng support; (3) bộ luật cuối cùng trùng khớp tuyệt đối giữa hai cài đặt độc lập; (4) kết quả có diễn giải kinh doanh rõ ràng, dẫn tới đề xuất hành động cụ thể.

Hạn chế: (1) phân tích chính giới hạn ở thị trường Anh (91,7% dữ liệu); các thị trường nhỏ không đủ dữ liệu cho ngưỡng support thống nhất; (2) mặt hàng xuất hiện dưới 30 giỏ bị loại, do đó không khai phá được luật cho các sản phẩm mới hoặc bán chậm; (3) luật được sinh trên tập “giỏ = đơn hàng” theo thời gian thực giao dịch, chưa tách các đơn lẻ của khách buôn khỏi đơn lẻ của khách lẻ; (4) thời gian thực nghiệm phụ thuộc cấu hình máy. Các hạn chế này là hướng cải thiện cho các đồ án tiếp theo.

KẾT LUẬN

Đồ án đã hoàn thành đầy đủ các yêu cầu của đề tài 19 với ba nhóm kết quả chính.

Về thuật toán: cài đặt hoàn chỉnh bằng Python hai thuật toán đã học - vét cạn và Apriori - với khả năng truy vết từng bước, kiểm chứng khớp 100% với đáp án chuẩn trong giáo trình Han và Kamber; tìm hiểu và ứng dụng thuật toán FP-Growth (chưa học) thông qua thư viện mlxtend, đồng thời dùng nó để kiểm chứng chéo với Apriori tự cài đặt trên dữ liệu thực. **Về thực nghiệm:** xây dựng trọn bộ quy trình xử lý dữ liệu thật gồm 541.909 dòng giao dịch - làm sạch, khám phá, xây dựng 16.406 giỏ hàng, mã hóa one-hot, khai phá ở 5 mức ngưỡng; tại $\text{min_support} = 2\%$ thu được 483 tập phổ biến và 202 luật kết hợp chất lượng ($\text{confidence} \geq 35\%$, $\text{lift} \geq 1,2$), số tập và số luật khớp tuyệt đối giữa hai cài đặt độc lập. **Về ứng dụng:** các luật được trực quan hóa đa góc nhìn (support - confidence, top lift, heatmap) và gom thành ba nhóm có ý nghĩa kinh doanh rõ ràng, kèm đề xuất cụ thể về gợi ý hoàn thiện bộ sưu tập, trưng bày liên sản phẩm, combo mùa vụ và nền tảng cho hệ thống gợi ý.

Hướng phát triển tiếp theo: tự cài đặt FP-Growth (cây FP) để so sánh cả ba cài đặt thuần tự viết; kết hợp phân khúc khách hàng RFM rồi khai phá luật riêng cho từng phân khúc; mở rộng dữ liệu sang các quốc gia khác với ngưỡng support thích ứng; và tích hợp bộ luật vào một ứng dụng gợi ý sản phẩm web đơn giản.

TÀI LIỆU THAM KHẢO

- [1] Agrawal, R., Imielinski, T., Swami, A. (1993). Mining association rules between sets of items in large databases. Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, 207-216.
- [2] Han, J., Kamber, M., Pei, J. (2012). Data Mining: Concepts and Techniques (3rd edition). Morgan Kaufmann Publishers.
- [3] Han, J., Pei, J., Yin, Y. (2000). Mining frequent patterns without candidate generation. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 1-12.
- [4] Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. Journal of Open Source Software, 3(24), 638.
- [5] Chen, S., Sguera, D. L., và cộng sự - UCI Machine Learning Repository (2019). Online Retail Data Set. <https://archive.ics.uci.edu/dataset/352/online+retail>
- [6] Tan, P.-N., Steinbach, M., Karpatne, A., Kumar, V. (2019). Introduction to Data Mining (2nd edition). Pearson.
- [7] Borgelt, C. (2003). Efficient implementations of Apriori and Eclat. Proceedings of the 1st International Workshop on Frequent Item Set Mining Implementations (FIMI 2003).
- [8] Tài liệu hướng dẫn sử dụng thư viện mlxtend - frequent patterns. http://rasbt.github.io/mlxtend/user_guide/frequent_patterns/

PHỤ LỤC

Phụ lục A. Hướng dẫn cài đặt và chạy chương trình

Chương trình được viết bằng Python 3.12 với các thư viện: pandas, matplotlib, openpyxl, mlxtend. Cài đặt và thực hiện:

```
# tạo môi trường ảo và cài thư viện
uv venv .venv -p 3.12 && source .venv/bin/activate
uv pip install pandas numpy matplotlib openpyxl mlxtend pypdf requests

# tải dữ liệu Online Retail (UCI) vào data/
# https://archive.ics.uci.edu/static/public/352/online+retail.zip

# chạy minh họa từng bước trên dữ liệu mẫu 9 giao dịch
python code/01_minh_hoa_tung_buoc.py

# chạy toàn bộ quy trình trên dữ liệu thực (~5 phút)
python code/02_phan_tich_du_lieu_lon.py
```

Kết quả chạy được ghi trong thư mục ket_qua/ (bảng tập phổ biến, bảng luật, bảng so sánh hiệu năng, tóm tắt JSON) và hình_anh/ (8 hình trong báo cáo). Toàn bộ mã nguồn đính kèm cùng báo cáo trong thư mục code/: thuat_toan.py (cài đặt thuật toán), 01_minh_hoa_tung_buoc.py (minh họa từng bước), 02_phan_tich_du_lieu_lon.py (quy trình dữ liệu thực).

Phụ lục B. Mã nguồn cài đặt thuật toán (thuat_toan.py)

```
# -*- coding: utf-8 -*-
"""
=====
DO AN CHUYEN NGANH - DE TAI 19
Ung dung khai pha luat ket hop (Association Rules) phan tich
hanh vi mua sam cua khach hang
=====
File: thuat_toan.py
Noi dung: Tu cai dat cac thuat toan khai pha tap pho bien va tao luat ket hop:
1. Brute-force (vet can)      - thuat toan co ban da hoc
2. Apriori                    - thuat toan trung tam da hoc (tu cai dat day du,
                               co buoc join + prune, co truy vet tung buoc)
3. Tao luat ket hop           - tu cai dat (confidence, lift, leverage, conviction)
4. FP-Growth                  - thuat toan CHUA HOC, goi tu thu vien mlxtend
                               (dung de so sanh hieu nang va kiem chung)
=====
"""

from itertools import combinations
from collections import defaultdict
import time
```

```

# =====
# 1. HAM PHU TRO CHUNG
# =====

def do_support_quet(giao_dich, ung_vien, n=None):
    """
    Quet toan bo giao dich de dem support cua tung tap ung vien (candidate).

    Tham so:
        giao_dich : list các tap (set/frozenset) - moi phan tu la 1 gio hang
        ung_vien   : list cac frozenset ung vien can dem
        n          : so giao dich (neu khong truyen se tu tinh)
    Tra ve:
        dict {frozenset: so_giao_dich_chua_tap_do}
    """
    if n is None:
        n = len(giao_dich)
    dem = {u: 0 for u in ung_vien}
    for t in giao_dich:
        t = frozenset(t)
        # tap rong luon la tap con cua moi giao dich
        for u in ung_vien:
            if u.issubset(t):
                dem[u] += 1
    return dem

def _lam_dep_label(tap):
    """Định dạng một tap item thành chuỗi '{a, b, c}' để in ra."""
    return "{" + ", ".join(sorted(tap)) + "}"

# =====
# 2. THUAT TOAN BRUTE-FORCE (VET CAN)
# =====
# Y tuong: xet TOAN BO cac tap con co the cua tap phan tu (ket hop toan hoc
#  $C(n,1) + C(n,2) + \dots = 2^n - 1$ ), dem support cua tung tap roi loc boi
# min_support. Do phuc tap mu, chi kha thi voi so luong phan tu nho.
# =====

def brute_force(giao_dich, min_sup, k_max=None, trace=None):
    """
    Khai pha tap pho bien bang thuat toan vet can.

    Tra ve: dict {k: {frozenset: so_lan_xuat_hien}} voi cac tap thoa min_sup.
    Neu truyen `trace` (list) thi tung buoc trung gian duoc ghi lai.
    """
    n = len(giao_dich)
    phan_tu = sorted({item for t in giao_dich for item in t})
    if k_max is None:
        k_max = len(phan_tu)

    ket_qua = {}
    if trace is not None:
        trace.append(("PHAN TU", phan_tu, None))

    for k in range(1, k_max + 1):

```

```

    # Buoc 1: sinh TAT CA tap con k phan tu (khong co prune)
    ung_vien = [frozenset(c) for c in combinations(phan_tu, k)]
    # Buoc 2: quet dem support tung ung vien
    dem = do_support_quet(giao_dich, ung_vien, n)
    # Buoc 3: loc boi min_support
    L_k = {u: c for u, c in dem.items() if c / n >= min_sup}
    if trace is not None:
        trace.append((f"C{k}", ung_vien, dict(dem)))
        trace.append((f"L{k}", L_k, None))
    if not L_k:
        break
    ket_qua[k] = L_k

return ket_qua

# =====
# 3. THUAT TOAN APRIORI (TU CAI DAT DAY DU)
# =====
# Tinh chat Apriori: moi tap con cua 1 tap pho bien cung pho bien
# => neu X khong pho bien thi moi tap lon hon chua X deu khong pho bien.
# Hai buoc lap voi k = 1, 2, ...:
#   - Join : sinh C(k) tu L(k-1) (F(k-1) x F(k-1))
#   - Prune : loai bo uong vien co tap con (k-1) phan tu nao do khong thuoc L(k-1)
#   - Quet : dem support, giu lai L(k)
# =====

def apriori(giao_dich, min_sup, trace=None):
    """
    Khai pha tap pho bien bang thuat toan Apriori (tu cai dat).

    Tham so:
        giao_dich : list tap item (moi giao dich 1 set)
        min_sup    : nguong ho tro toi thieu (0 < min_sup <= 1)
        trace      : neu la list, tung buoc se duoc ghi de phuc vu trinh bay
    Tra ve:
        (F, dem_L1) voi F = {k: {frozenset: dem}}, dem theo SO LUOT xuat hien.
    """
    n = len(giao_dich)
    G = [frozenset(t) for t in giao_dich]

    # ----- Buoc 1: tim L1 -----
    dem1 = defaultdict(int)
    for t in G:
        for item in t:
            dem1[item] += 1
    L1 = {frozenset([i]): c for i, c in dem1.items() if c / n >= min_sup}
    if trace is not None:
        trace.append(("C1", [frozenset([i]) for i in sorted(dem1)],
                     {frozenset([i]): c for i, c in dem1.items()}))
        trace.append(("L1", L1, None))
    if not L1:
        return {}, dem1

    F = {1: L1}
    k = 2
    while F.get(k - 1):
        F_km1 = F[k - 1]

```

```

tap_F_km1 = set(F_km1.keys())

# ----- Buoc 2 (JOIN): sinh C(k) tu F(k-1) -----
# chuan hoa moi tap (k-1) thanh tuple sap xep de join theo thu tu
tap_sap_xep = sorted([sorted(u) for u in tap_F_km1])
ung_vien = set()
for i in range(len(tap_sap_xep)):
    for j in range(i + 1, len(tap_sap_xep)):
        a, b = tap_sap_xep[i], tap_sap_xep[j]
        if a[:-1] == b[:-1]: # giong nhau k-2 phan tu dau
            u = frozenset(a) | frozenset(b)
            if len(u) == k:
                ung_vien.add(u)

# ----- Buoc 3 (PRUNE): loai ung vien co tap con khong pho bien -----
C_k, bi_cat = [], []
for u in ung_vien:
    if all(frozenset(sub) in tap_F_km1 for sub in combinations(u, k - 1)):
        C_k.append(u)
    else:
        bi_cat.append(u)

# ----- Buoc 4 (QUET): dem support cua C(k) -----
dem = do_support_quet(G, C_k, n)
L_k = {u: c for u, c in dem.items() if c / n >= min_sup}

if trace is not None:
    trace.append((f"C{k}", C_k, dict(dem), bi_cat))
    trace.append((f"L{k}", L_k, None))

if L_k:
    F[k] = L_k
    k += 1

return F, dem1

# =====
# 4. APRIORI TOI UU CHO DU LIEU LON (dem theo danh sach giao dich dung chung)
# =====
# Van la dung thuat toan Apriori, nhung buoc dem support khong quet lai toan
# bo giao dich voi tung ung vien, ma dung chi muc nguoc (inverted index):
# item -> tap hop cac giao dich chua item.
# Support cua tap X = kich thuoc giao cua cac danh sach giao dich cua item.
# =====

def apriori_nhanh(giao_dich, min_sup, max_k=None):
    n = len(giao_dich)
    G = [frozenset(t) for t in giao_dich]
    chi_muc = defaultdict(set) # item -> tap tid
    for tid, t in enumerate(G):
        for item in t:
            chi_muc[item].add(tid)

    dem1 = {i: len(tids) for i, tids in chi_muc.items()}
    L1 = {frozenset([i]): c for i, c in dem1.items() if c / n >= min_sup}
    if not L1:

```

```

        return {}

F = {1: L1}
# tid-list của các tap pho bien hien tai
tid_lists = {u: set.intersection(*(chi_muc[i] for i in u)) for u in L1}
k = 2
while F.get(k - 1) and (max_k is None or k <= max_k):
    F_km1 = F[k - 1]
    tap_F_km1 = set(F_km1.keys())
    tap_sap_xep = sorted([sorted(u) for u in tap_F_km1])
    ung_vien = set()
    for i in range(len(tap_sap_xep)):
        for j in range(i + 1, len(tap_sap_xep)):
            a, b = tap_sap_xep[i], tap_sap_xep[j]
            if a[:-1] == b[:-1]:
                ung_vien.add(frozenset(a) | frozenset(b))

    L_k = {}
    for u in ung_vien:
        if not all(frozenset(sub) in tap_F_km1 for sub in combinations(u, k - 1)):
            continue # prune
        # dem support bang giao tid-list: chi tinh tren cac tap con da biet
        tids = set.intersection(*(tid_lists[frozenset(sub)]
                                   for sub in combinations(u, k - 1)))
        if len(tids) / n >= min_sup:
            L_k[u] = len(tids)
            tid_lists[u] = tids
    if L_k:
        F[k] = L_k
    k += 1
return F

# =====
# 5. TAO LUAT KET HOP TU CAC TAP PHO BIEN (TU CAI DAT)
# =====
# Voi moi tap pho bien X ( $|X| \geq 2$ ) va moi phan hoach  $X = A \cup B$  (A, B khác rỗng):
#   support(X) = dem(X) / N
#   confidence(A->B) = support(X) / support(A)
#   lift(A->B) = confidence / support(B)
#   leverage(A->B) = support(X) - support(A)*support(B)
#   conviction(A->B) = (1 - support(B)) / (1 - confidence)
# Chi giu luat thoa min_conf (va min_lift neu co).
# =====

def tao_luat_ket_hop(F, n, min_conf=0.5, min_lift=1.0):
    """
    F : dict {k: {frozenset: dem}} ket qua khai pha tap pho bien
    n : so giao dich
    Tra ve: list dict luat, moi luat gom: truoc (A), sau (B), va cac do luong.
    """
    luat = []
    for k, L_k in F.items():
        if k < 2:
            continue
        for X, dem_X in L_k.items():
            sup_X = dem_X / n
            cac_phan_tu = sorted(X)

```

```

        for r in range(1, k):
            for A in combinations(cac_phan_tu, r):
                A = frozenset(A)
                B = X - A
                dem_A = _dem_tap(F, A)
                dem_B = _dem_tap(F, B)
                if not dem_A or not dem_B:
                    continue
                sup_A, sup_B = dem_A / n, dem_B / n
                conf = sup_X / sup_A
                lift = conf / sup_B
                if conf < min_conf or lift < min_lift:
                    continue
                lev = sup_X - sup_A * sup_B
                conv = (1 - sup_B) / (1 - conf) if conf < 1 else float("inf")
                luat.append({
                    "truoc": A, "sau": B,
                    "sup_X": sup_X, "sup_A": sup_A, "sup_B": sup_B,
                    "conf": conf, "lift": lift,
                    "leverage": lev, "conviction": conv,
                })
    luat.sort(key=lambda d: (-d["lift"], -d["conf"]))
    return luat

def _dem_tap(F, u):
    """Tra ve so luot xuat hien cua tap u trong ket qua Apriori F (0 neu khong co)."""
    for k in (len(u),):
        L_k = F.get(k, {})
        if u in L_k:
            return L_k[u]
    return 0

def thoi_gian(ham, *a, **kw):
    """Do thoi gian chay cua 1 ham, tra ve (ket_qua, so_giay)."""
    t0 = time.perf_counter()
    kq = ham(*a, **kw)
    return kq, time.perf_counter() - t0

```